МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01. «Математика и компьютерные науки»

Отчет о выполнении проектной работы

По дисциплине: Цифровая аналитика
На тему: «Генератор резюме»

Выполнил студент
группы 5130201/40002 _____ Храмов В.Д.

Принял
к. т. н. _____ Мулюха В.А.

«___» _____ 2026 г.

Санкт-Петербург
2026

Содержание

1 Техническое задание 3

2 История запросов 5

1 Техническое задание

Проект представляет собой веб-приложение, предназначенное для автоматического создания резюме в формате PDF на основе данных, введённых пользователем через веб-форму.

Функциональные возможности

Приложение собирает основные персональные данные, информацию о навыках, образовании и опыте работы. Поддерживается загрузка фотографии и выбор одного из нескольких шаблонов оформления. Перед генерацией итогового файла система предоставляет предварительный просмотр документа в браузере.

Технологический стек: Backend-часть реализована на языке Go с использованием стандартного HTTP-сервера и HTML-шаблонов.

Вводимые пользователем данные

- ФИО и контактные данные (телефон, email, город);
- краткая информация о себе;
- образование (учебное заведение, специальность, годы обучения);
- опыт работы (компания, должность, период, обязанности);
- навыки и дополнительная информация (языки, сертификаты);
- фотография профиля.

Требования и ограничения

Данные и текст:

- Текстовые поля: до 1000 символов для больших блоков, до 100 символов для коротких полей.
- Поддерживаемые языки: русский и английский (кодировка UTF-8).

Ограничения на фотографию:

- Форматы: JPG, JPEG, PNG.
- Максимальный размер файла: 5 МБ.
- Минимальное разрешение: 400×400 px.

Оформление и шрифты:

- Используются стандартные шрифты (Arial, Times New Roman).
- В PDF применяются встроенные шрифты для корректного отображения на любых устройствах.
- Размеры шрифтов фиксированы шаблоном.

Интерфейс и шаблоны

Интерфейс разделен на две области: слева — форма ввода, справа — динамический предпросмотр, который обновляется автоматически.

Доступные шаблоны:

1. Классический — чёрно-белая цветовая схема, традиционная структура.
2. Современный — цветовые акценты, визуальные разделители и иконки.

2 История запросов

Диалог 1

Запрос (промт):

мне нужно сделать проект, вот описание того, как он должен выглядеть

Проект представляет собой веб-приложение «Генератор резюме», предназначенное для автоматического создания резюме в формате pdf на основе данных, введённых пользователем через веб-форму. Информация от пользователя: основные персональные данные, навыки, образование и опыт работы. Также приложение будет поддерживать загрузку фотографии пользователя и выбор одного из нескольких шаблонов оформления резюме. После заполнения формы система будет давать предварительный просмотр документа в браузере, позволяя пользователю проверить корректность данных и внешний вид перед генерацией итогового файла. После генерации pdf будет происходить отправка готового файла пользователю для скачивания.
Backend-часть приложения будет реализована на языке Go с использованием стандартного http-сервера и шаблонов html
Вводимые пользователем данные
Пользователь заполняет следующие данные:
ФИО
Контактные данные (телефон, email, город)
Краткая информация о себе
Образование (учебное заведение, специальность, годы обучения)
Опыт работы (компания, должность, период, обязанности)
Навыки
Дополнительная информация (языки, сертификаты и т.д.)
Фотография

Ограничения на данные
Текстовые поля: до 1000 символов для больших блоков, до 100 символов для ↪ коротких полей.
Поддерживаемые языки: русский и английский (символьная кодировка UTF-8).

Ограничения на фотографию
Форматы: JPG, JPEG, PNG
Максимальный размер файла: 5 МБ
Минимальное разрешение: 400×400 px
Соотношение сторон: любое

Шрифты и оформление
Используются стандартные веб-безопасные шрифты (Arial, Times New Roman)
В PDF применяются встроенные шрифты для корректного отображения на любых устройствах (Arial, Times New Roman)
Размеры шрифтов фиксированы шаблоном, пользователь их не меняет.

Предварительный внешний вид приложения
Интерфейс состоит из двух областей:
слева — форма ввода данных
справа — динамический предпросмотр резюме
Предпросмотр обновляется автоматически после изменения полей, чтобы пользователь сразу видел результат.

Шаблоны резюме
Планируется реализовать несколько шаблонов:
Классический — чёрно-белая цветовая схема, традиционная структура блоков.
Современный — использование цветовых акцентов, визуальных разделителей, иконок навыков.

мне нужно сделать проект, вот описание того, как он должен выглядеть

Проект представляет собой веб-приложение «Генератор резюме», предназначенное для автоматического создания резюме в формате pdf на основе данных, введённых пользователем через веб-форму. Информация от пользователя: основные персональные данные, навыки, образование и опыт работы. Также приложение будет поддерживать загрузку фотографии пользователя и выбор одного из нескольких шаблонов оформления резюме. После заполнения формы система будет давать предварительный просмотр документа в браузере, позволяя пользователю проверить корректность данных и внешний вид перед генерацией итогового файла. После генерации pdf будет происходить отправка готового файла пользователю для скачивания.
Backend-часть приложения будет реализована на языке Go с использованием стандартного http-сервера и шаблонов html
Вводимые пользователем данные
Пользователь заполняет следующие данные:
ФИО
Контактные данные (телефон, email, город)
Краткая информация о себе
Образование (учебное заведение, специальность, годы обучения)
Опыт работы (компания, должность, период, обязанности)
Навыки

Дополнительная информация (языки, сертификаты и т.д.)
Фотография

Ограничения на данные
Текстовые поля: до 1000 символов для больших блоков, до 100 символов для коротких полей.
Поддерживаемые языки: русский и английский (символьная кодировка UTF-8).

Ограничения на фотографию
Форматы: JPG, JPEG, PNG
Максимальный размер файла: 5 МБ
Минимальное разрешение: 400×400 px
Соотношение сторон: любое

Шрифты и оформление
Используются стандартные веб-безопасные шрифты (Arial, Times New Roman)
В PDF применяются встроенные шрифты для корректного отображения на любых устройствах (Arial, Times New Roman)
Размеры шрифтов фиксированы шаблоном, пользователь их не меняет.

Предварительный внешний вид приложения
Интерфейс состоит из двух областей:
слева — форма ввода данных
справа — динамический предпросмотр резюме
Предпросмотр обновляется автоматически после изменения полей, чтобы пользователь сразу видел результат.

Шаблоны резюме
Планируется реализовать несколько шаблонов:
Классический — чёрно-белая цветовая схема, традиционная структура блоков.
Современный — использование цветовых акцентов, визуальных разделителей, иконок навыков.

Ответ:

Сделаю минимально-полный каркас проекта на Go: страница с двухпанельным UI (форма слева + live-предпросмотр справа), поддержка фото/валидаций/двух шаблонов, и генерация PDF через headless Chrome (PrintToPDF). Сейчас посмотрю, что уже есть в папке и разверну структуру проекта.

Папка проекта пустая — создам всё с нуля: `go.mod`, сервер, шаблоны, статику и
↪ README с инструкцией запуска. Сначала проверю/прочитаю будущий
↪ `README.md`, чтобы дальше аккуратно править файлы.

Сейчас поправлю зависимости: уберу “угаданные” версии из `go.mod` и поставлю
↪ актуальные через `go get`, чтобы всё собиралось гарантированно.

Я добавил основной Go-сервер и эндпоинты (UI, preview, PDF, раздача загруженной
↪ фото). Дальше добавлю HTML-шаблоны и фронтенд JS/CSS, затем приведу
↪ код к компилируемому виду (поправлю embed/static и зависимости) и проверю
↪ сборку/запуск.

Теперь доведу сервер до “собирается и работает”: поправлю embed-раздачу статики,
↪ добавлю `FuncMap` для современного шаблона (разбиение навыков на
↪ “чипсы”), и заменю самописные хелперы на стандартные (`bytes.NewReader`,
↪ `url.PathEscape/QueryEscape`, `fs.Sub`). После этого поставлю зависимости и
↪ проверю `go run`.

Дальше — поставлю зависимости (`go mod tidy`) и проверю, что сервер
↪ компилируется и стартует. Если всплывут ошибки
↪ (импорты/шаблоны/вёрстка), сразу поправлю.

Зависимости подтягиваются, но `chromedp` потребовал более свежий Go toolchain
↪ (Go 1.26) — `go` сейчас автоматически скачивает нужный toolchain. Подожду
↪ завершения `go mod tidy`, затем проверю сборку и запуск сервера.

`go mod tidy` завис из-за скачивания Go toolchain: в песочнице запрещена запись в
↪ системные кэши Go. Сейчас остановлю текущий процесс и запущу снова вне
↪ песочницы, чтобы Go смог установить нужный toolchain и зависимости.

Проблема оказалась не в коде, а в доступе к `proxy.golang.org` (TLS timeout). Я
↪ переключил загрузку модулей на прямое скачивание из GitHub
↪ (`GOPROXY=direct`) и жду, пока `go mod tidy` докачает зависимости.

Отлично: зависимости подтянулись. Сейчас `go test ./...` докачивает нужный Go
↪ toolchain (Go 1.26) и затем соберёт проект — это выявит все ошибки
↪ компиляции/импортов, которые я тут же исправлю.

Сборка прошла успешно, сервер запущен. Сейчас проверю, что он действительно
↪ слушает порт и отдаёт главную страницу/статические файлы/preview, чтобы UI
↪ работал как задумано (две панели + live preview).

Сейчас всё работает, но `chromedp` в свежей версии тянет Go 1.26 toolchain. Чтобы
↪ проект был проще для сдачи (без автоскачивания нового Go), попробую
↪ закрепить `chromedp` на версии, совместимой с Go 1.22, и снова проверю
↪ генерацию PDF.

Диалог 2

Запрос (промт):

пропало соединенние, продолжай работу

Ответ:

Продолжаю. Сейчас закреплю зависимости `chromedp` на версии, которая не
↪ требует Go 1.26 (чтобы проект спокойно собирался на Go 1.22), затем заново
↪ прогоню сборку и проверю генерацию PDF.

Вижу, что `go.mod` уже переключился на `go 1.26`, и `cdproto` стоит свежий —
↪ именно он тянет требование Go 1.26. Сейчас подберу более старую версию
↪ `cdproto`, совместимую с Go 1.22, зафиксирую её и верну `go 1.22` в `go.mod`,
↪ затем снова проверю сборку и PDF.

Ок, я убрал зависимость от `chromedp`/`cdproto` (из-за требования Go 1.26) и
↪ перевёл генерацию PDF на вызов установленного Chrome/Chromium в
↪ headless-режиме. Теперь верну `go.mod` на Go 1.22, удалю лишние зависимости,
↪ сделаю `go mod tidy` и повторно проверю: предпросмотр и скачивание PDF.

Отлично: теперь проект снова собирается на Go 1.22 и сервер запущен. Дальше
↪ быстро проверю, что предпросмотр работает, и что PDF генерируется через
↪ Chrome/Chromium (если браузер не найден — будет понятная ошибка и можно
↪ указать `CHROME_BIN`).

Готово — проект собран и работает

Я продолжил работу и собрал полноценный каркас приложения **«Генератор
↪ резюме»** на **Go (стандартный `net/http` + HTML templates)**:

- **UI**: 2 панели — слева форма, справа **live-предпросмотр** (обновляется автоматически).
- **Данные**: ФИО, контакты, о себе, навыки, образование (повторяемые блоки), опыт (повторяемые блоки), доп. инфо, **фото**.
- **Валидации**:
  - короткие поля до **100** символов, длинные блоки до **1000**
  - фото **JPG/JPEG/PNG**, до **5 МБ**, минимум **400×400**
- **Шаблоны**: `classic` и `modern`.
- **PDF**: генерируется через установленный **Chrome/Chromium headless** и отдается пользователю на скачивание.

Где что лежит

- **Сервер**: `cmd/server/main.go`
- **UI**: `cmd/server/web/templates/ui.html`
- **Шаблоны резюме**:
  - `cmd/server/web/templates/resume_classic.html`
  - `cmd/server/web/templates/resume_modern.html`
- **Статика**: `cmd/server/web/static/app.css`, `cmd/server/web/static/app.js`
- **Инструкция**: `README.md`

Как запустить

```bash
go mod tidy
go run ./cmd/server
```

Открыть: `http://127.0.0.1:8080`

Если PDF не генерируется из-за того, что Chrome не найден — укажи путь:

```bash
CHROME_BIN="/path/to/chrome" go run ./cmd/server
```

Диалог 3

Запрос (промт):

не удается загрузить фото, оно не отображается в резюме

Ответ:

Понял — фото загружается, но в предпросмотре/резюме не отображается. Чаще всего это из-за защитных заголовков (браузер блокирует загрузку картинки в `iframe srcdoc`). Сейчас поправлю заголовки, чтобы `/uploads/...` корректно подгружалось в предпросмотре, и сразу проверю.

Код поправлен. Сейчас перезапущу сервер, чтобы новые заголовки применились, и после этого фото в предпросмотре должно начать отображаться.

Порт 8080 занят — значит старый сервер ещё запущен. Сейчас найду процесс, который слушает 8080, остановлю его и запущу сервер заново.

Причина и фикс

Фото не отображалось из-за слишком строгого заголовка **`Cross-Origin-Resource-Policy: same-origin`**: предпросмотр рендерится в `iframe srcdoc`, и некоторые браузеры воспринимают его как “другой/opaque origin”, из-за чего картинка по URL `/uploads/...` блокировалась.

Я исправил это: теперь сервер отдаёт

- **`Cross-Origin-Resource-Policy: cross-origin`**

вместо `same-origin`.

Изменение в файле: `cmd/server/main.go` (функция `wrap`).

Что сделать тебе

- Перезапусти сервер:

```bash
go run ./cmd/server
```

- Обнови страницу (лучше с жёстким обновлением) и попробуй снова загрузить фото — оно должно появиться в резюме и в PDF.

Если после перезапуска всё равно не видно фото, напиши:
- какой браузер,
- и загружается ли фото вообще (ошибка/нет),
и я добавлю быстрый лог/диагностику прямо в UI (покажу причину
↪ блокировки/валидации).

Диалог 4

Запрос (промт):

придумай тестовый пример резюме, который я могу загрузить и проверить
↪ генерацию резюме, чтобы не вводить каждый раз руками данные.

Ответ:

Сделаю, чтобы ты мог одним кликом подставлять тестовые данные: добавлю кнопку
↪ **«Заполнить пример»** в интерфейс и встроенный тестовый набор
↪ (ФИО/контакты/skills/education/experience/доп. инфо). После клика форма
↪ заполнится, предпросмотр обновится автоматически, и можно сразу жать
↪ «Скачать PDF».

Добавил кнопку **«Заполнить пример»** и встроенный тестовый набор данных.
↪ Сейчас останется просто перезапустить сервер и проверить в браузере: нажал
↪ кнопку → форма заполнилась → предпросмотр обновился → PDF скачался.

Готово: тестовый пример в один клик

Я добавил в интерфейс кнопку **«Заполнить пример»** — она автоматически
↪ подставляет тестовые данные (ФИО, контакты, о себе, навыки, 2 записи
↪ образования, 2 записи опыта, доп. инфо) и сразу обновляет предпросмотр.

Как пользоваться
- Перезапусти сервер (если он уже запущен — просто перезапусти, чтобы
↪ подтянулась новая статика).
- Открой `http://127.0.0.1:8080`
- Нажми **«Заполнить пример»**

- При желании выбери фото вручную (его нельзя проставить автоматически из JS
↪ из-за ограничений браузера)
- Нажми **«Скачать PDF»**

Если хочешь, могу сделать ещё вариант: загрузка примера из файла `example.json`,
↪ чтобы ты мог редактировать данные без правки JS.

Диалог 5

Запрос (промт):

давай

Ответ:

Ок — сделаю вариант с внешним файлом `example.json`, чтобы ты мог менять
↪ тестовые данные без правки кода. Я добавлю `static/example.json` и переключу
↪ кнопку **«Заполнить пример»** на загрузку этого файла (с запасным
↪ встроенным примером, если файл не прочитается).

Сделано: теперь пример берётся из отдельного файла `example.json`, который
↪ можно редактировать. Сейчас перезапущу сервер, чтобы браузер точно взял
↪ свежую статику, и ты сразу сможешь нажать «Заполнить пример».

Сделал: пример из файла `example.json`

Теперь кнопка **«Заполнить пример»** берёт данные не из JS, а из файла:

- `cmd/server/web/static/example.json`

Ты можешь менять этот JSON как хочешь — и UI будет подставлять новые данные.

Как проверить
1) Перезапусти сервер (если ещё не перезапускал после обновлений):

```bash
go run ./cmd/server
```

```

2) Открой `http://127.0.0.1:8080` → нажми **«Заполнить пример»**.

### Важно про фото
Фото **нельзя** подставить автоматически из файла (ограничение браузера на `input type="file"`), его нужно выбрать вручную, но все остальные поля подставятся.
```

Диалог 6

Запрос (промт):

```
у меня все равно не работает загрузка фото, убери ограничения, если они есть, так чтобы примерно такие фото могли быть загружены и корректно отображались в резюме
```

Ответ:

```
Ок, уберу/ослаблю ограничения на фото так, чтобы “паспортные” снимки вроде твоего PNG проходили: расширю допустимые форматы, уберу минимальное разрешение (или сильно снижу), и сделаю валидацию более мягкой. Сначала посмотрю реальный размер/тип твоего файла, чтобы точно попасть в причину отказа.

Нашёл причину на твоём примере: у файла размер **354×472**, а у нас стояло требование **минимум 400×400**, поэтому загрузка отклоняется. Сейчас уберу проверку на минимальное разрешение (и сделаю определение формата по фактическому декодированию), чтобы такие фото загружались и отображались.

Ограничение по минимальному разрешению убрано — теперь фото 354×472 и подобные должны проходить. Перезапущу сервер, затем проверю загрузку на твоём файле (через запрос к `/api/preview` с фото) и убедимся, что URL картинки отдаётся и она доступна.

### Исправил: фото теперь проходит и отображается
```

Причина была в ограничении **минимум 400×400** — твоё фото **354×472**,
↪ поэтому оно отклонялось.

Что я сделал в `cmd/server/main.go`:

- **убрал проверку минимального разрешения**
- сделал определение типа картинки по **реальному декодированию** (JPEG/PNG),
↪ а не по расширению/`Content-Type` (чтобы “странно сохранённые” файлы тоже
↪ проходили)

Сервер я перезапустил и проверил твоим файлом — `/api/preview` вернул
↪ корректный `src="/uploads/..."`, а `/uploads/...` отдаёт `200 OK` и нужные
↪ заголовки, так что в предпросмотре картинка должна отображаться.

Что тебе сделать
1) Перезапусти сервер:

```bash
go run ./cmd/server
```

2) Открой страницу, выбери фото и посмотри предпросмотр.

Если в браузере всё ещё не видно фото — скажи, **какой браузер**, и скинь текст
↪ ошибки из блока статуса (если появляется).

Диалог 7

Запрос (промт):

добавь еше три разных шаблона резюме , чтобы их общее количество было 5

Ответ:

Ок, добавлю ещё **3 шаблона**, чтобы стало **5**: расширю список в UI, добавлю
↪ 3 HTML-шаблона, и обновлю Go-сервер так, чтобы он рендерил выбранный
↪ шаблон (с фоллбеком на `classic`). После этого проверю сборку и что
↪ предпросмотр/PDF работают со всеми 5 вариантами.

Я перевёл сервер на карту шаблонов (чтобы легко расширять до 5+) и сейчас
↪ добавлю сами 3 новых HTML-шаблона + обновлю валидацию выбора шаблона и
↪ выпадающий список в UI. Затем прогоню сборку и быстро проверю preview/pdf
↪ для каждого нового шаблона.

Шаблоны добавлены и сервер перезапущен. Сейчас быстро проверю, что
↪ `/api/preview` возвращает HTML для каждого из 5 шаблонов (и что PDF тоже
↪ генерируется). Если где-то упадёт — сразу поправлю.

Сделал 5 шаблонов (добавил ещё 3)

Добавлены новые шаблоны резюме, теперь доступно **5 вариантов**:

- `classic` (Классический)
- `modern` (Современный)
- `minimal` (Минималистичный)
- `sidebar` (Сайдбар)
- `elegant` (Элегантный)

Что изменилось в проекте
- **UI**: выпадающий список шаблонов расширен до 5
↪ (`cmd/server/web/templates/ui.html`)
- **Backend**: сервер теперь хранит шаблоны в
↪ `map[TemplateID]*template.Template` и рендерит по `template` из формы
↪ (`cmd/server/main.go`)
- **Новые файлы шаблонов**:
  - `cmd/server/web/templates/resume_minimal.html`
  - `cmd/server/web/templates/resume_sidebar.html`
  - `cmd/server/web/templates/resume_elegant.html`
- **README** обновил под 5 шаблонов

Как проверить
Открой `http://127.0.0.1:8080`, выбери разные шаблоны в селекте — предпросмотр и
↪ PDF должны меняться.

Диалог 8

Запрос (промт):

замени шаблон минималистичный на шаблон яндкса, котрый я приложил

Ответ:

Ок — заменю шаблон `minimal` на стиль “Яндекса” по твоему макету: зелёные плашки-заголовки, двухколоночная сетка (слева образование/опыт/другое, справа навыки), крупный блок “РЕЗЮМЕ”, фото справа сверху. Сейчас обновлю `resume_minimal.html`, и добавлю небольшой хелпер в Go, чтобы в заголовке “НАПРАВЛЕНИЕ/ДОЛЖНОСТЬ” автоматически подставлялась должность (из первого места работы).

Я почти закончил замену `minimal` на “Яндекс”-макет. Сейчас быстро поправлю заголовок “НАПРАВЛЕНИЕ/ДОЛЖНОСТЬ”, чтобы он корректно подставлял первую должность без лишней/дублирующей логики, затем перезапущу сервер и проверю предпросмотр/PDF именно на шаблоне `minimal`.

Готово: шаблон `minimal` заменён на “Яндекс” из твоего макета

Я полностью **переписал** `cmd/server/web/templates/resume_minimal.html` под стиль как на картинке:

- **зелёные плашки** “РЕЗЮМЕ” и “НАПРАВЛЕНИЕ/ДОЛЖНОСТЬ”
- верхний блок: **ФИО + контакты слева**, **фото справа**
- основная часть: **2 колонки** (слева образование/опыт/другое, справа навыки со списком)
- “НАПРАВЛЕНИЕ/ДОЛЖНОСТЬ” теперь **автоматически берётся из первой должности** в опыте (если есть)

Также в UI я заменил название пункта:
- значение осталось `minimal`, но отображается как **«Яндекс»** (`cmd/server/web/templates/ui.html`)

Что нужно сделать тебе
- Обнови страницу (лучше hard refresh) и выбери шаблон **«Яндекс»**.
- Нажми **«Заполнить пример»** (можешь в `example.json` поставить `"template": "minimal"` если хочешь, чтобы пример сразу открывался в этом стиле).

Если хочешь, могу ещё ближе “попасть” в макет: добавить декоративные элементы (иконки/полоски) как на скрине, но без картинок это будет именно стилизация через CSS.

Диалог 9

Запрос (промт):

удали шаблон яндекс

Ответ:

Ок — удалю шаблон **«Яндекс»** (это `template=minimal`): уберу его из выпадающего списка, из списка разрешённых шаблонов и из карты шаблонов на сервере, а файл `resume_minimal.html` удалю. Затем проверю сборку и перезапущу сервер.

Сделано: шаблон «Яндекс» удалён

- **Убрал** опцию `minimal` (Яндекс) из списка шаблонов в UI (`cmd/server/web/templates/ui.html`)
- **Удалил** поддержку `TemplateMinimal` на бэкенде (из разрешённых шаблонов и карты рендера) (`cmd/server/main.go`)
- **Удалил файл** шаблона `cmd/server/web/templates/resume_minimal.html`
- **Обновил README**: теперь шаблонов **4** (`classic`, `modern`, `sidebar`, `elegant`)
- Проект **собирается**, сервер запущен на `http://127.0.0.1:8080`

Если хочешь снова иметь 5 шаблонов — скажи, какой стиль сделать вместо удалённого (например: “Европейский/Europass”, “Тёмный”, “Инфографика”, “Деловой с линиями”), и я добавлю новый.

Диалог 10

Запрос (промт):

придумай два тестовых примера, которые смогут протестировать все ограничение программы (на русском и на англисйком языках) и сохрани их так, чтобы я мог загрузить и проверить генерацию резюме

Ответ:

Сделаю 2 тестовых примера, которые проверяют ограничения: один на русском, второй на английском, и оба будут **валидными на границах** (короткие поля близко к 100 символам, длинные блоки близко к 1000). Сохраню их как отдельные JSON в `static/` и добавлю в UI две кнопки, чтобы подставлять их в форму одним кликом.

Сейчас добавлю два файла `example_ru_limits.json` и `example_en_limits.json` (валидные, близко к лимитам), и расширю UI: две отдельные кнопки для RU/EN. Кнопки будут подставлять соответствующий файл, чтобы ты мог быстро прогонять предпросмотр и PDF без ручного ввода.

Добавил два “лимитных” примера и кнопки в UI. Сейчас перезапущу сервер, проверю что оба JSON отдаются как статика, и что предпросмотр/PDF успешно генерируются с каждым примером.

Файлы RU/EN примеров раздаются корректно. Сейчас прогоню полноценный тест: возьму каждый JSON, отправлю как форму в `/api/preview` и `/api/pdf` и проверю, что оба успешно отрабатывают (это подтвердит, что лимитные строки не превышают ограничения).

Сделал 2 тестовых примера (RU/EN) для проверки ограничений

Я добавил два JSON-примера, которые **проверяют ограничения** (короткие поля близко к 100 символам, большие блоки близко к 1000, переносы строк, UTF-8 RU/EN) и **успешно проходят** предпросмотр и генерацию PDF.

Файлы
- `cmd/server/web/static/example_ru_limits.json`
- `cmd/server/web/static/example_en_limits.json`

Как загрузить в форме
В UI появились 2 кнопки:
- **«Пример RU (лимиты)»**
- **«Example EN (limits)»**

Нажимаешь кнопку → форма заполняется → предпросмотр обновляется → жмёшь **«Скачать PDF»**.

Примечание: фото, как и раньше, нужно выбрать вручную (браузер не даёт
↪ подставлять файл программно).

Диалог 11

Запрос (промт):

нужно во всех шаблонах увеличить размер фото.

Ответ:

Ок — увеличу размер фото во всех шаблонах одинаково и аккуратно: подниму
↪ размеры контейнера/изображения (где есть) и подправлю отступы, чтобы
↪ вёрстка не “ломалась”. Сейчас найду, где в шаблонах задаются классы
↪ `.photo`/`photoBox` и обновлю.

Нашёл размеры фото в каждом шаблоне. Сейчас увеличу их везде до **140×140** (и
↪ чуть подправлю скругление там, где нужно), затем быстро проверю сборку и
↪ перезапущу сервер, чтобы ты сразу увидел результат в предпросмотре и PDF.

Готово — увеличил фото во всех шаблонах

Во всех 4 шаблонах, где есть фото, увеличил размер до **140×140**:

- `cmd/server/web/templates/resume_classic.html`
- `cmd/server/web/templates/resume_modern.html`
- `cmd/server/web/templates/resume_sidebar.html`
- `cmd/server/web/templates/resume_elegant.html`

Сервер перезапущен, можешь обновить страницу и проверить предпросмотр/PDF.

Диалог 12

Запрос (промт):

нужно исправить название сгенерированного файла-резюме на ФИО введенным
↪ пользователем

Ответ:

Сейчас это почти сделано, но для русских букв многие браузеры игнорируют
↪ `filename="..."` и дают своё имя. Исправлю заголовок `Content-Disposition` по
↪ стандарту: добавлю `filename*` (UTF-8), а на фронте научу скачивание брать
↪ имя из `filename*`. Тогда файл будет называться **точно по ФИО** (включая
↪ кириллицу).

Я проверю заголовок `Content-Disposition` корректной командой (с `-X POST` не
↪ нужен, `-F` сам делает POST) и убедимся, что там есть `filename*` с UTF-8.
↪ После этого скачивание в браузере будет именовать PDF по введённому ФИО.

Исправил имя скачиваемого PDF на ФИО пользователя

Сделал корректный `Content-Disposition` для **кириллицы**:

- сервер теперь отдаёт **`filename*` (UTF-8)** + ASCII-fallback `filename`
- фронт при скачивании **сначала читает `filename*`**, затем `filename`

Изменено:
- `cmd/server/main.go` — `Content-Disposition` теперь через
↪ `contentDispositionPDF(...)`
- `cmd/server/web/static/app.js` — парсинг `filename*`

Проверка заголовка показывает, что теперь имя будет таким:
`Иванов_Иван_Иванович.pdf` (по введённому ФИО).

Диалог 13

Запрос (промт):

в сгенерированных файлах нужно убрать все пометки, которые не касаются
↪ пользователя, то есть 20.04.2026, 19:30 Resume file:///var/folders/_x/gc11_cs92
↪ 11fw24bfkby8z_80000gn/T/resume-pdf-1617910078/resume.html и тп

Ответ:

Понял: это стандартные **header/footer** от печати Chrome (дата/время, заголовок “Resume”, `file:///...`). Их нужно отключить на этапе генерации PDF. Сейчас проверю, какие флаги поддерживает ваш Chrome/Chromium, и поправлю команду генерации так, чтобы PDF был “чистым”, без служебных пометок.

Сейчас перезапущу сервер и быстро сгенерирую PDF, чтобы убедиться: вверху/внизу страницы больше нет даты, названия “Resume” и `file://...` — то есть header/footer печати действительно выключены.

Отлично: `file:///...` и путь `/var/folders...` уже убрались (значит header/footer печати отключились). Оставшееся слово “Resume” сейчас приходит из `<title>Resume</title>` внутри HTML-шаблонов (это не пометка пользователя). Сейчас поменяю `<title>` во всех шаблонах на ФИО пользователя, чтобы никаких “Resume” больше не появлялось.

Убрал все “служебные” пометки из PDF.

- **Отключил header/footer печати Chrome** (дата/время, URL, путь `file:///...` и т.п.) — в генерации PDF добавлены флаги `--no-pdf-header-footer` и (для старых версий) `--print-to-pdf-no-header` в `cmd/server/main.go`.
- **Убрал “Resume”** как не относящееся к пользователю: во всех шаблонах `<title>` теперь **ФИО пользователя** (или “Резюме”, если ФИО пустое):
  - `cmd/server/web/templates/resume_classic.html`
  - `cmd/server/web/templates/resume_modern.html`
  - `cmd/server/web/templates/resume_sidebar.html`
  - `cmd/server/web/templates/resume_elegant.html`

Я сгенерировал PDF и проверил — строк **`Resume`**, **`file://`**, **`var/folders`**, **`20.04.2026`** там больше нет.

Диалог 14

Запрос (промт):

нужно сдеать, чтобы шаблоны «современный» и «элегантный» были во весь лист с минимальными рамками белого листа.

Ответ:

Ок — сделаю так, чтобы шаблоны **«Современный»** (`modern`) и **«Элегантный»** (`elegant`) занимали почти весь лист: уменьшу поля печати (`@page margin`) и уберу ограничение по ширине (`max-width`), чтобы контент тянулся на весь лист. Затем проверю генерацию PDF.

Готово — `modern` и `elegant` теперь “во весь лист” с минимальными полями

В шаблонах **«Современный»** (`resume_modern.html`) и **«Элегантный»** (`resume_elegant.html`) я:

- уменьшил поля печати: `@page { margin: 6mm; }`
- убрал внешние отступы страницы: `body { margin: 0; }`
- снял ограничение по ширине: `.wrap { max-width: none; width: 100%; margin: 0; }`

Изменённые файлы:
- `cmd/server/web/templates/resume_modern.html`
- `cmd/server/web/templates/resume_elegant.html`

Сервер перезапущен — обнови страницу и посмотри предпросмотр/PDF: белые рамки должны стать минимальными.

Диалог 15

Запрос (промт):

создай gitignore, подготовь проект, чтобы его можно было выложить на github. укажи в readme что проект сделан с испольщованием тебя

создай gitignore, подготовь проект, чтобы его можно было выложить на github. укажи в readme что проект сделан с испольщованием тебя

Ответ:

Сделаю: добавлю корректный `.gitignore` для Go/OS/IDE и обновлю `README` (инструкция перед публикацией + пометка, что проект сделан с использованием меня). После этого быстро проверю, что ничего лишнего не попадает в репозиторий.

Готово.

Сделал подготовку проекта к публикации на GitHub:

- Добавил `.gitignore` с нужными исключениями:
  - macOS: `.DS_Store`
  - артефакты Go: `bin/`, `dist/`, `*.out`, `coverage.out`, и т.д.
  - временные/локальные: `tmp/`, `.tmp/`, `*.log`
  - IDE: `.idea/`, `.vscode/`, swap-файлы

- Обновил `README.md`:
  - добавил раздел **«Публикация на GitHub»** с шагами `git add/commit/push`
  - добавил пометку, что проект разработан с использованием AI-ассистента Codex (Cursor)
  - поправил описание ограничения фото (сейчас у вас уже без минимального разрешения)

Также проверил сборку: `go test ./...` проходит.

Если хочешь, следующим шагом могу сразу подготовить аккуратный `LICENSE` (например, MIT) и шаблон `CONTRIBUTING.md`.

Диалог 16

Запрос (промт):

придумай еще 6 разных шаблонов для резюме, шаблоны не должны поторять друг друга и предыдущие.

Ответ:

Отлично, сделаю: добавлю **6 новых, визуально разных** шаблонов и подключу их в систему выбора шаблонов (UI + backend), чтобы они работали и в предпросмотре, и в PDF. Сначала обновлю сервер/селект, затем создам 6 отдельных HTML-шаблонов.

Добавляю 6 новых шаблонов как отдельные файлы: `nordic`, `terminal`, `academic`, `magazine`, `infographic`, `monochrome` — у каждого своя композиция и стиль, чтобы они не повторяли уже существующие.

Проверка API прошла, но чтобы гарантированно убедиться, что выбранные именно новые шаблоны (а не fallback), я сейчас перезапущу сервер и проверю уникальные маркеры каждого из 6 шаблонов в HTML-предпросмотре.

Готово — добавил **6 новых шаблонов**, все разные и не повторяют текущие:

1. `nordic` — холодная палитра, градиентный hero-блок, карточная сетка
2. `terminal` — тёмный “консольный” стиль (моноширинный)
3. `academic` — строгий академический стиль с хронологией по периодам
4. `magazine` — журнальная верстка с “обложкой” и асимметричной колонкой
5. `infographic` — инфографичный формат с круглым фото и “прогресс-барами” навыков
6. `monochrome` — контрастный ч/б стиль с жесткой типографикой

Что изменил

- Подключил новые `TemplateID` и рендер на backend: `cmd/server/main.go`
- Добавил шаблоны в выбор на UI: `cmd/server/web/templates/ui.html`
- Создал 6 новых файлов:
  - `cmd/server/web/templates/resume_nordic.html`
  - `cmd/server/web/templates/resume_terminal.html`
  - `cmd/server/web/templates/resume_academic.html`
  - `cmd/server/web/templates/resume_magazine.html`
  - `cmd/server/web/templates/resume_infographic.html`
  - `cmd/server/web/templates/resume_monochrome.html`

Проверка

- `go test ./...` — проходит
- предпросмотр для всех новых шаблонов отрабатывает корректно

Если хочешь, следующим шагом могу под каждый из этих 6 шаблонов добавить
↪ отдельные тестовые JSON-примеры (чтобы в один клик смотреть “лучший”
↪ внешний вид каждого).

Диалог 17

Запрос (промт):

в шаблоне терминал нужно чтобы углы резюме были закруглены, как сделан
↪ реальный терминал на macos

в шаблоне academic в пердпросмотре слишком маленький левый край, нужно его
↪ увеличить.

Ответ:

Сделаю обе правки:
1) для `terminal` — оформлю окно как macOS terminal (более выраженное
↪ скругление и аккуратный внешний контур),
2) для `academic` — увеличу левый отступ/поле в предпросмотре, чтобы край не
↪ “прилипал”.
После правок проверю сборку.

Готово, внес обе правки:

- В `terminal` (`cmd/server/web/templates/resume_terminal.html`)
  - увеличил скругление окна до `16px`
  - добавил более “macOS-подобную” глубину через `box-shadow`
  - визуально карточка теперь ближе к окну реального Terminal на macOS

- В `academic` (`cmd/server/web/templates/resume_academic.html`)
  - добавил дополнительный левый внутренний отступ для всего содержимого:
    ↪ `padding-left: 8mm`
  - обернул контент в контейнер `.page`, чтобы отступ применялся стабильно

Проверка: `go test ./...` проходит.
Обнови страницу предпросмотра — изменения уже должны быть видны.

Диалог 18

Запрос (промт):

исправь readme, убери все что не нужно, оставь только описание проекта, как его запускать

Ответ:

Сделаю README минимальным: оставлю только краткое описание проекта и инструкции по запуску. Сейчас очищу лишние разделы и оставлю рабочий “как запустить”.

Готово — `README.md` почистил и оставил только нужное:

- описание проекта
- требования
- как запускать

Убрал все дополнительные разделы (ограничения, тестовые примеры, публикация, примечания).

Диалог 19

Запрос (промт):

добавь в @README.md подпись что проект сделан с твоей помощью

Ответ:

Добавлю короткую подпись в конец `README.md`, что проект сделан с моей помощью.

Готово, добавил подпись в `README.md`:

`Проект сделан с помощью AI-ассистента Codex (Cursor).`